

Assignment Report

Sneha Kumari (Roll No: 241024)

March 2026

1 Overview of the Problem

The goal of this assignment is to design a memory-efficient system in C++ that builds a word-level index over large text files. The primary challenge is to process these files without loading the entire content into memory, ensuring that memory usage is independent of the file size].

2 Design and Approach

The solution follows an **Object-Oriented Design** (OOD) approach. I implemented an abstract base class for queries to utilize **runtime polymorphism** and dynamic dispatch. The system processes files incrementally using a fixed-size buffer to maintain a low memory footprint.

3 Description of the Classes Used

According to the design requirements, the following classes were implemented:

- **FileProcessor**: Handles the buffered file reading logic and manages the 256 KB to 1024 KB fixed-size buffer.
- **Tokenizer**: Extracts alphanumeric characters and ensures case-insensitive word matching.
- **Indexer**: Manages the word-level mapping (word $\rightarrow$ frequency) for different versions.
- **Query Classes**: An abstract base class **Query** with derived classes **WordQuery** and **TopKQuery** to process different analytical requests.

4 How the File is Processed Using the Fixed-Size Buffer

The program allocates a buffer of a constant size throughout execution. Data is read into the buffer in chunks. To handle tokens split across buffer boundaries, any partial word at the end of a buffer is stored and prepended to the next chunk before tokenization resumes.

5 Any Assumptions or Observations

- **Assumptions:** It is assumed that no single word exceeds the minimum buffer size of 256 KB.
- **Observations:** Memory usage grows only with the number of unique words found. The implementation of **function overloading** and **templates** ensured the code remains modular and reusable.